

Pre-Sales AI Agent — Technical Implementation (Concise)

Field	Value
Version	2.0 (concise export)
Stack	FastAPI 2.0 + React 2.0
Full reference	docs/TECHNICAL_IMPLEMENTATION_FULL.md · docs/DATA_FLOW_DIAGRAM.md

1. Overview

Enterprise pre-sales RFP automation: upload TXT/PDF/DOCX → multi-agent AI pipeline → requirements, summary, scope, clarifying questions → user answers → **final 15-section proposal** + timeline emails.

Tier	Technology
Backend	Python 3.11+, FastAPI, SQLite, JWT, optional SMTP
Frontend	React 18, TypeScript, Vite, Fluent UI
AI	Groq (default), Ollama, Azure OpenAI, or mock
RAG	In-memory or ChromaDB chunk retrieval

Two-phase pipeline: Phase 1 runs automatically (no draft). Phase 2 generates the proposal after POST /rfp/{id}/clarifications/resolve.

2. Architecture & Flow

Layers: React SPA (:5173) → FastAPI (:8000) → Orchestrator → Specialist agents → LLM + vector store + SQLite.

Phase 1 (auto): Ingest (20%) → Understanding (40%) → Summarization (60%) → Scope (70%) → Clarification (80%) → Complete (100%, chat opened).

Phase 2 (user): Submit answers → RAG + ResponseDraftAgent → timeline email.

Component	Path	Role
App	backend/main.py	CORS, lifespan, routers
Orchestrator	backend/orchestrator/pipeline.py	Agent sequence, RUNS/OUTPUTS
RFP API	backend/routers/rfp.py	Upload, status, output, chat, resolve

Component	Path	Role
LLM	backend/core/llm.py	chat_json / chat_text, JSON repair
Config	backend/core/config.py	.env at repo root
Persistence	backend/services/run_store.py	data/runs.sqlite3

3. Technology & Layout

Backend: FastAPI, Uvicorn, Pydantic v2, httpx, python-jose, bcrypt, pypdf, python-docx, APScheduler, agent-framework-core.

Frontend: React, Vite, Fluent UI, React Router, Zustand (auth), Axios.

```
presales-agent/
├── backend/      main.py, core/, routers/, orchestrator/, agents/, models/, r
ag/, services/
├── frontend/    src/app, pages, api/client.ts
├── specs/       Spec Kit YAML (not loaded at runtime)
├── data/        uploads/, runs.sqlite3, chroma/, email_outbox/
└── docs/        This doc + full reference + data-flow diagrams
```

4. Configuration (Essential)

Loaded from repo-root .env via backend/core/config.py. Paths resolve relative to repo root.

Variable	Purpose
LLM_BACKEND	groq ollama azure_openai mock
GROQ_API_KEY, GROQ_MODEL	Cloud LLM (default: llama-3.3-70b-versatile)
GROQ_INTER_REQUEST_DELAY_SECONDS	Groq TPM cooldown between agents (55s+ for 70B)
LLM_ONLY_MODE	Block mock for real runs
VECTOR_STORE_BACKEND	memory or chroma
JWT_SECRET, JWT_EXPIRES_MINUTES	Auth (change secret in prod)
EMAIL_ENABLED, SMTP_*	Timeline + reminder emails
DATA_DIR, RUN_DB_PATH	Uploads and SQLite
REMINDER_DAYS_BEFORE	Deadline reminder offset (default 3)

See `.env.example` and full doc for complete list.

5. Security & Auth

- **JWT Bearer** on all `/api/v1/rfp/*`; bcrypt passwords in users table.
 - **assert_run_access**: per-user run isolation; history filtered by `user_id`.
 - **Production**: rotate `JWT_SECRET`, restrict CORS, never commit `.env`, validate uploads.
 - **LLM data**: content sent to configured provider; use Ollama for on-premises.
-

6. Agents (Summary)

Agent	Module	LLM	Output
Ingestion	<code>ingestion_agent.py</code>	No	RFPDocument, chunks
Understanding	<code>understanding_agent.py</code>	Yes	Requirement[] (REQ-###, MAF, ambiguity)
Summarization	<code>summarization_agent.py</code>	Yes	RFPSummary
Scope	<code>scope_gathering_agent.py</code>	Yes	PresalesScopeDocument
Clarification	<code>clarification_agent.py</code>	Yes	ClarifyingQuestion[]
Response Draft	<code>response_agent.py</code>	Yes	15-section proposal (phase 2)
Evaluation	<code>evaluation_agent.py</code>	Yes	MAF audit (tool / future loop)
Presales Chat	<code>presales_chat_agent.py</code>	Yes	Clarification assistant

Agent Framework tools: `ingest_document`, `retrieve_context`, `audit_draft`, `revise_draft`.

Draft agent: Single FINAL-PROPOSAL section with Cover Page through Conclusion; uses RAG snippets + `clarification_resolution_document`; JSON → repair → prose fallback.

7. LLM & RAG

Client: `OpenAICompatibleClient` — `chat_json` for agents, `chat_text` for prose fallback.

JSON pipeline: JSON-only suffix → provider `json_object / format`: `json` → fence stripping, truncation repair, optional retry.

Errors: LLMError with hints for 429/413/timeouts on Groq.

Vector store: memory (substring match) or chroma (persistent). Chunks upserted after ingestion; draft queries top-k (8) snippets.

8. Data & Persistence

SQLite (data/runs.sqlite3): runs (status, progress, user_id), run_outputs (JSON OutputBundle), users, chat messages, reminders.

Files: data/uploads/{uuid}.ext, optional data/chroma/, data/email_outbox/ when SMTP off.

Hydration: hydrate_caches() on startup restores RUNS/OUTPUTS from SQLite.

Key models: RFPDocument, Requirement, RFPSummary, OutputBundle, DraftResponse, TimelineMilestone — see backend/models/.

9. API Reference

Base: <http://127.0.0.1:8000/api/v1> · Swagger: /docs

Method	Path	Description
POST	/auth/register, /auth/login	Account + JWT
GET	/auth/me	Current user
GET	/health	{ status, server_boot_id }
POST	/rfp/analyze	Upload RFP → { run_id }
GET	/rfp/{id}/status	Progress / error
GET	/rfp/{id}/output	Full bundle
GET	/rfp/{id}/summary, /questions, /draft, /scope	Subsets
GET/POST	/rfp/{id}/chat	Clarification messages
POST	/rfp/{id}/clarifications/ resolve	Answers + files → final draft
GET	/rfp/history	User run list
DELETE	/rfp/{id}	Delete run

Method	Path	Description
WS	/rfp/{id}/stream	Progress events

Status codes: 401 auth, 403 wrong user, 404 not ready, 502 LLM failure.

10. Frontend

Route	Page
/login, /register	Auth
/dashboard, /intake	Home, upload
/analysis/:runId	Poll status
/results/:runId	Summary, reqs, questions, draft, timeline
/agents, /run/:runId/agents/:id	Agent workspace
/history	Past runs

Auth: Zustand + localStorage; Axios Bearer interceptor; AuthSessionGuard detects backend restart via server_boot_id.

Dev proxy: Vite /api → 127.0.0.1:8000.

11. Notifications

Event	When
Timeline email	After final draft (notify_draft_complete)
Reminder	REMINDER_DAYS_BEFORE before each milestone

APScheduler polls SCHEDULER_INTERVAL_SECONDS; SMTP or data/email_outbox/.

12. Deployment

Local: uvicorn main:app --reload (backend) + npm run dev (frontend). Set GROQ_API_KEY in .env.

Production: Single uvicorn worker (in-memory RUNS per process); nginx proxy /api → backend; npm run build for static frontend.

Environment	LLM	Vector	Email
Dev	groq/ollama	memory	outbox
Prod	azure_openai/groq	chroma	SMTP

13. Troubleshooting

Symptom	Fix
Stuck / slow pipeline	Groq rate limit — wait, increase GROQ_INTER_REQUEST_DELAY_SECONDS
LLMError / bad JSON	Use llama-3.3-70b-versatile
Empty draft	Complete phase 2; avoid 8B models for large RFPs
404 after restart	Re-login; verify run exists in SQLite for your user
No email	Check EMAIL_ENABLED, SMTP, or data/email_outbox/

Tests: `pytest tests/ -q` (backend), `npm test` (frontend).

14. References

Resource	Location
Full technical doc	<code>docs/TECHNICAL_IMPLEMENTATION_FULL.md</code>
Data flow diagrams	<code>docs/DATA_FLOW_DIAGRAM.md</code>
Quick start	<code>README.md</code>
OpenAPI	<code>http://127.0.0.1:8000/docs</code>

*Concise export — regenerate PDF/DOCX via python
`scripts/generate_technical_docs.py`*